In [10]:
```
# %pip install pandas numpy matplotlib seaborn scikit-learn scipy openpyxl p
```

In [11]:
```
# %pip install opencv-python
```

In [ ]:
```

```

In [110…
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error
```

In [111…
```
data = pd.read_csv('uber.csv')
```

In [112…
```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
 #   Column             Non-Null Count   Dtype  
---  ------             --------------   -----  
 0   Unnamed: 0         200000 non-null  int64  
 1   key                200000 non-null  str    
 2   fare_amount        200000 non-null  float64
 3   pickup_datetime    200000 non-null  str    
 4   pickup_longitude   200000 non-null  float64
 5   pickup_latitude    200000 non-null  float64
 6   dropoff_longitude  199999 non-null  float64
 7   dropoff_latitude   199999 non-null  float64
 8   passenger_count    200000 non-null  int64  
dtypes: float64(5), int64(2), str(2)
memory usage: 13.7 MB
```

In [113…
```
data =data.drop(columns=['Unnamed: 0', 'key'])
```

In [114…
```
data.shape
```

Out[114…
```
(200000, 7)
```

In [115…
```
data.columns
```

Out[115…
```
Index(['fare_amount', 'pickup_datetime', 'pickup_longitude', 'pickup_latitu
de',
       'dropoff_longitude', 'dropoff_latitude', 'passenger_count'],
      dtype='str')
```

In [116…
```
data.describe()
```

Out[116…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	2000
count	200000.000000	200000.000000	200000.000000	199999.000000	199999.000000		
mean	11.359955	-72.527638	39.935885	-72.525292	39.923890		
std	9.901776	11.437787	7.720539	13.117408	6.794829		
min	-52.000000	-1340.648410	-74.015515	-3356.666300	-881.985513		
25%	6.000000	-73.992065	40.734796	-73.991407	40.733823		
50%	8.500000	-73.981823	40.752592	-73.980093	40.753042		
75%	12.500000	-73.967154	40.767158	-73.963658	40.768001		
max	499.000000	57.418457	1644.421482	1153.572603	872.697628		2

In [117…
```
data.isnull().sum()
```

Out[117…
```
fare_amount          0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    1
dropoff_latitude     1
passenger_count      0
dtype: int64
```

In [118…
```
data['dropoff_longitude'] = data['dropoff_longitude'].fillna(data['dropoff_l
```

In [119…
```
data['dropoff_latitude'] = data['dropoff_latitude'].fillna(data['dropoff_lat
```

In [120…
```
data.isnull().sum()
```

Out[120…
```
fare_amount          0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    0
dropoff_latitude     0
passenger_count      0
dtype: int64
```

In [121…
```
data['pickup_datetime'] = pd.to_datetime(data['pickup_datetime'], errors='co
```

In [122…
```
data = data.dropna(subset=['pickup_datetime'])
```

In [123…
```
data['hour'] = data['pickup_datetime'].dt.hour
data['day'] = data['pickup_datetime'].dt.day
```

```
data['month'] = data['pickup_datetime'].dt.month
data['year'] = data['pickup_datetime'].dt.year
data['day_of_week']=data['pickup_datetime'].dt.day_of_week

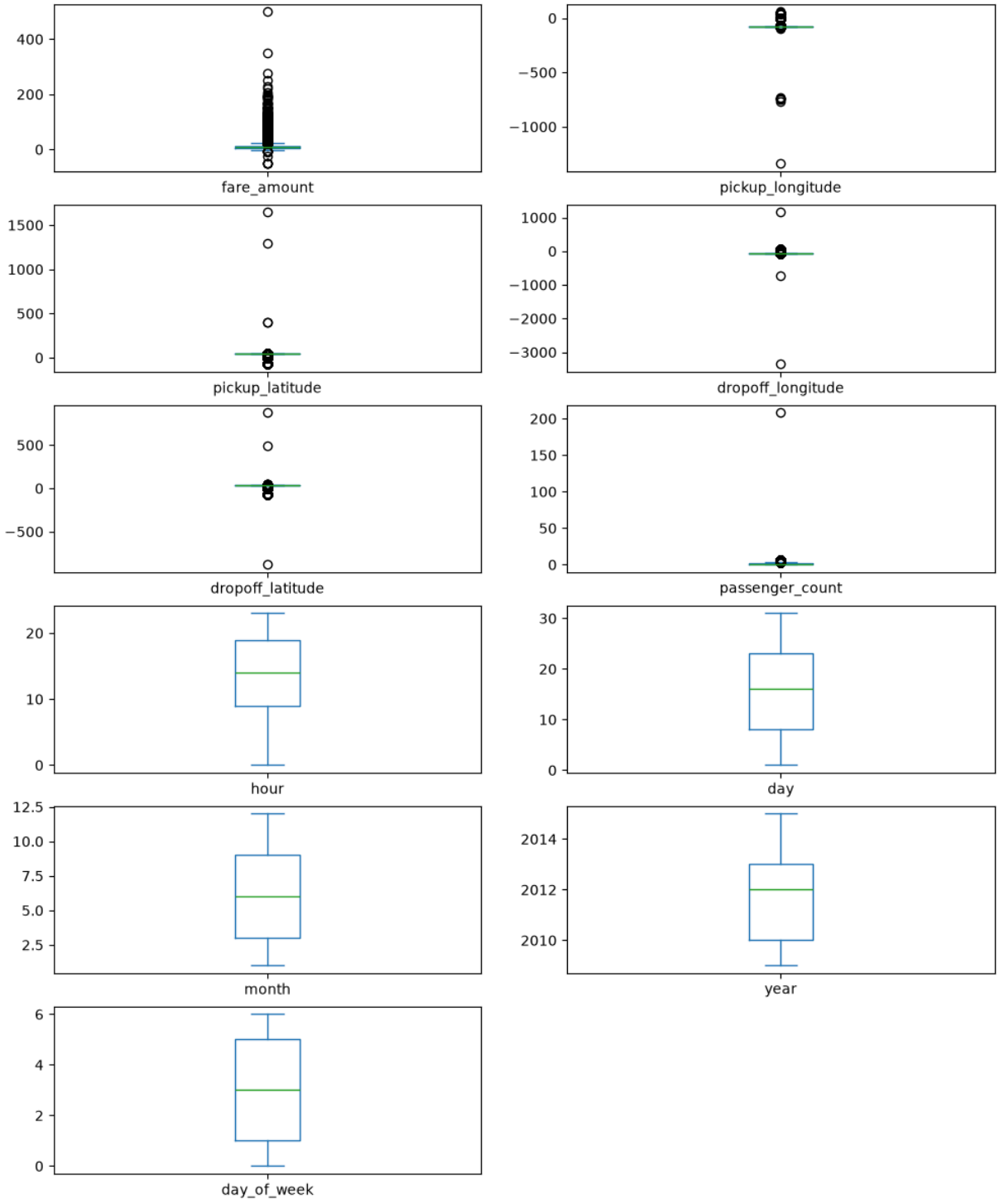
data = data.drop(columns=['pickup_datetime'])
```

In [124…
```
data.columns
```

Out[124…
```
Index(['fare_amount', 'pickup_longitude', 'pickup_latitude',
       'dropoff_longitude', 'dropoff_latitude', 'passenger_count', 'hour',
       'day', 'month', 'year', 'day_of_week'],
      dtype='str')
```

In [125…
```
data.plot(kind = 'box', subplots = True, layout = (6,2), figsize=(12,15))
```

Out[125…
```
fare_amount             Axes(0.125,0.77;0.352273x0.11)
pickup_longitude      Axes(0.547727,0.77;0.352273x0.11)
pickup_latitude         Axes(0.125,0.638;0.352273x0.11)
dropoff_longitude     Axes(0.547727,0.638;0.352273x0.11)
dropoff_latitude        Axes(0.125,0.506;0.352273x0.11)
passenger_count       Axes(0.547727,0.506;0.352273x0.11)
hour                    Axes(0.125,0.374;0.352273x0.11)
day                   Axes(0.547727,0.374;0.352273x0.11)
month                   Axes(0.125,0.242;0.352273x0.11)
year                  Axes(0.547727,0.242;0.352273x0.11)
day_of_week             Axes(0.125,0.11;0.352273x0.11)
dtype: object
```



In [126…

```
def remove_outlier(data,col):
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)

    IQR = Q3-Q1

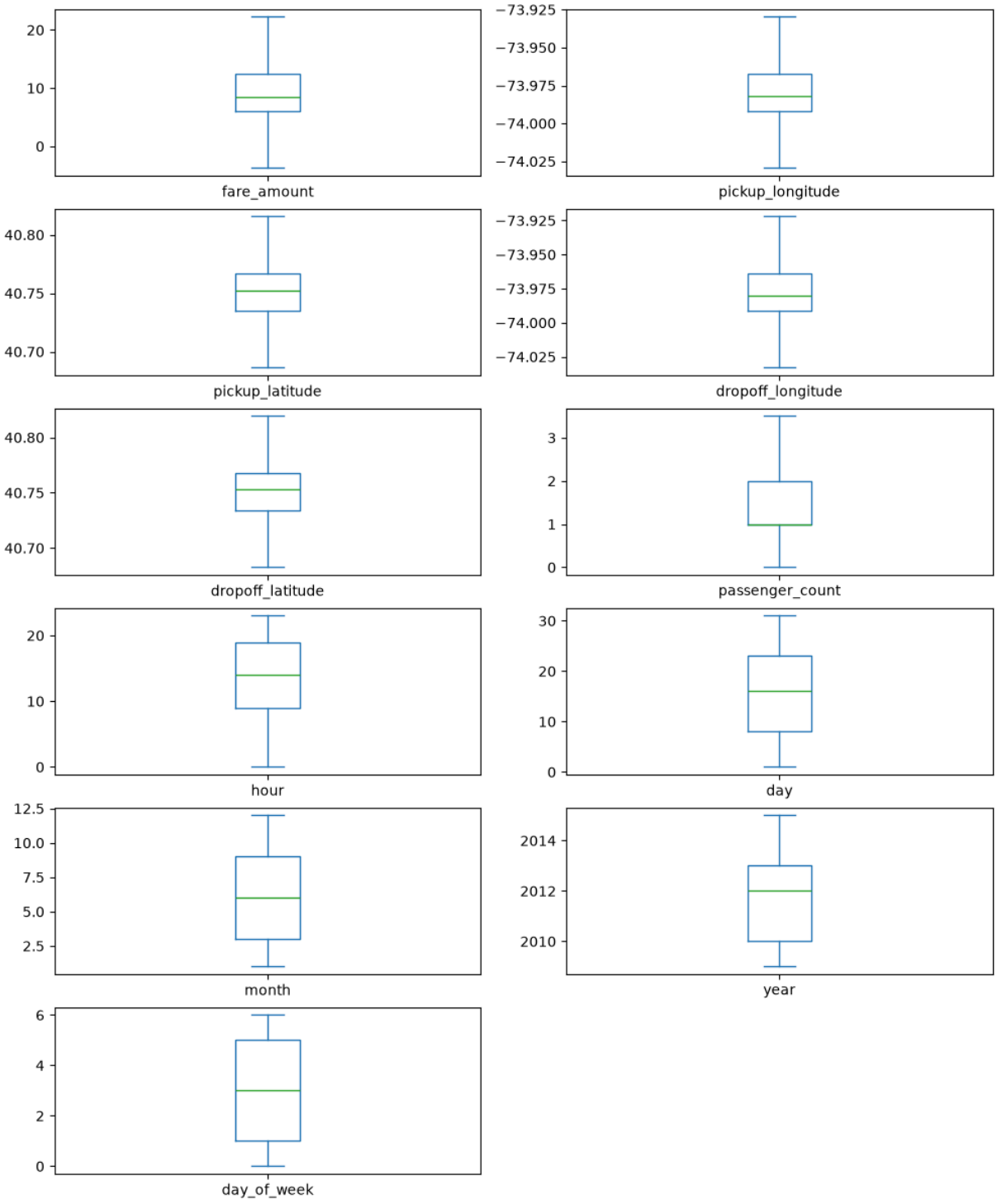
    upper_wiskers = Q3 + 1.5*IQR
    lower_wiskers = Q1 - 1.5*IQR

    data[col] = np.clip( data[col], lower_wiskers, upper_wiskers)
    return data
```

In [127…
```
for col in data.select_dtypes(include='number').columns:
    data = remove_outlier(data,col)
```

In [128…
```
data.plot(kind='box' , subplots=True, layout=(6,2), figsize=(12,15))
```

Out[128…
```
fare_amount            Axes(0.125,0.77;0.352273x0.11)
pickup_longitude      Axes(0.547727,0.77;0.352273x0.11)
pickup_latitude        Axes(0.125,0.638;0.352273x0.11)
dropoff_longitude     Axes(0.547727,0.638;0.352273x0.11)
dropoff_latitude       Axes(0.125,0.506;0.352273x0.11)
passenger_count       Axes(0.547727,0.506;0.352273x0.11)
hour                   Axes(0.125,0.374;0.352273x0.11)
day                   Axes(0.547727,0.374;0.352273x0.11)
month                  Axes(0.125,0.242;0.352273x0.11)
year                  Axes(0.547727,0.242;0.352273x0.11)
day_of_week            Axes(0.125,0.11;0.352273x0.11)
dtype: object
```

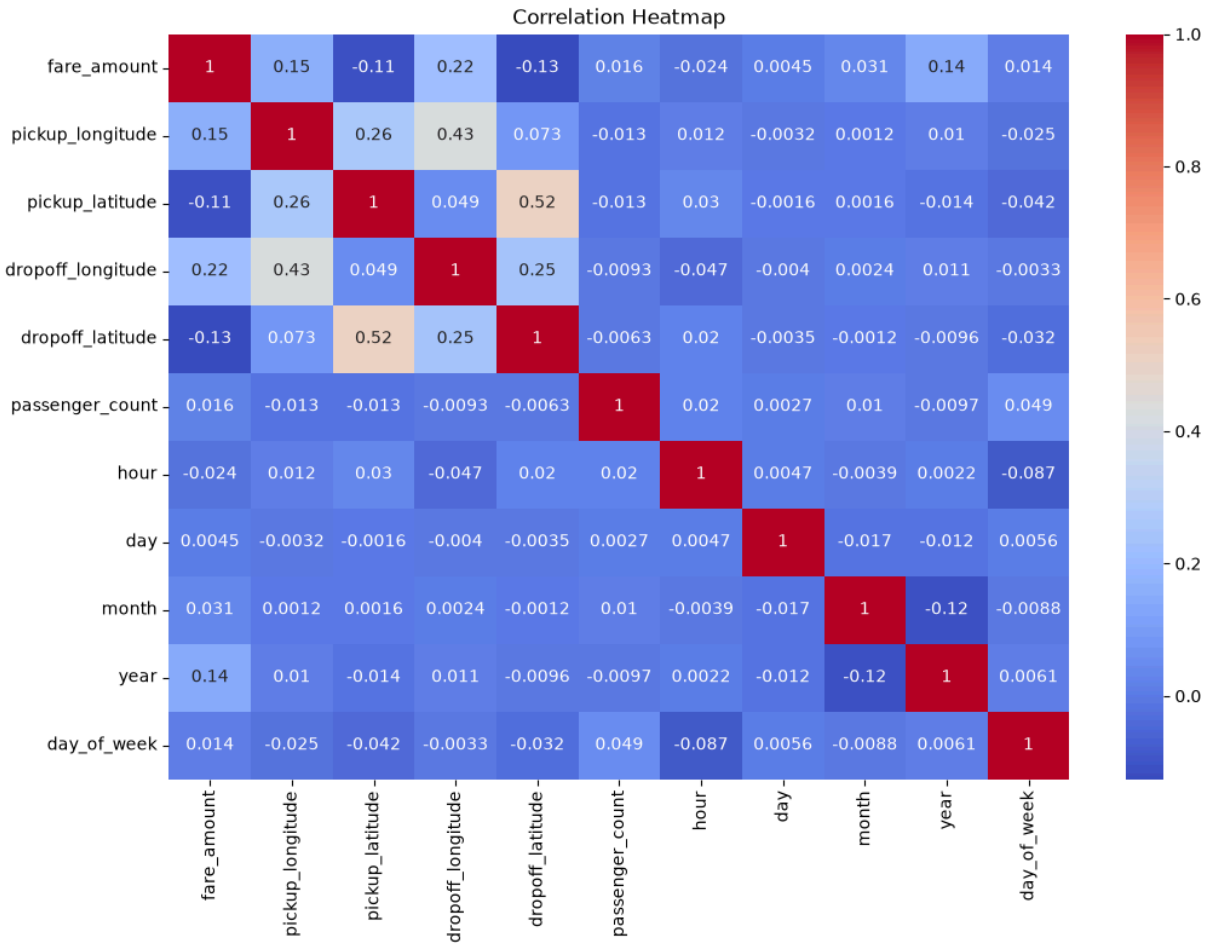


In [129…
```
data.shape
```

Out[129…
```
(200000, 11)
```

In [134…
```
plt.figure(figsize=(12, 8))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
```

Out[134…
```
Text(0.5, 1.0, 'Correlation Heatmap')
```



In [135…
```
lat1 = np.radians(data['pickup_latitude'])
lon1 = np.radians(data['pickup_longitude'])
lat2 = np.radians(data['dropoff_latitude'])
lon2 = np.radians(data['dropoff_longitude'])

dlat = lat2 - lat1
dlon = lon2 - lon1

a = (np.sin(dlat / 2)**2 +
     np.cos(lat1) * np.cos(lat2) *
     np.sin(dlon / 2)**2)

a = np.clip(a, 0, 1)

data['dist_travel_km'] = 2 * 6371 * np.arcsin(np.sqrt(a))
```

In [136…
```
# 12. SELECT FEATURES AND TARGET
X = data.drop(columns=['fare_amount'])
y = data['fare_amount']
```

In [137…
```
# 13. TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42
)
```

In [138…
```
# 14. LINEAR REGRESSION
lr = LinearRegression()
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)

print("\nLINEAR REGRESSION")
print("Intercept:", lr.intercept_)
print("Coefficients:", lr.coef_)
print("R2 Score:", r2_score(y_test, lr_pred))
print("MSE:", mean_squared_error(y_test, lr_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, lr_pred)))
```

```
LINEAR REGRESSION
Intercept: 3673.537215360576
Coefficients: [ 2.59832974e+01 -6.93677247e+00  1.96236415e+01 -1.82884732e+
01
  6.29021730e-02  7.07686483e-03  4.30981789e-03  5.90438206e-02
  3.64185590e-01 -3.01335384e-02  1.85061540e+00]
R2 Score: 0.6581801196247368
MSE: 10.120416265844746
RMSE: 3.1812601694681852
```

In [139…
```
# 15. RANDOM FOREST REGRESSION
rf = RandomForestRegressor(
    n_estimators=10, random_state=42, n_jobs=-1
)

rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

print("\nRANDOM FOREST REGRESSION")
print("R2 Score:", r2_score(y_test, rf_pred))
print("MSE:", mean_squared_error(y_test, rf_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, rf_pred)))
```

```
RANDOM FOREST REGRESSION
R2 Score: 0.7716927178498845
MSE: 6.759597274875316
RMSE: 2.5999225517071305
```

In [140…
```
# 16. COMPARE BOTH MODELS
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Random Forest'],
    'R2 Score': [
        r2_score(y_test, lr_pred),
        r2_score(y_test, rf_pred)
    ],
    'MSE': [
        mean_squared_error(y_test, lr_pred),
        mean_squared_error(y_test, rf_pred)
    ],
    'RMSE': [
        np.sqrt(mean_squared_error(y_test, lr_pred)),
        np.sqrt(mean_squared_error(y_test, rf_pred))
    ]
})
```

```
print("\nMODEL COMPARISON")
print(results.to_string(index=False))

print("\nBest Model:",
      results.loc[results['RMSE'].idxmin(), 'Model'])
```

```
MODEL COMPARISON
            Model  R2 Score       MSE     RMSE
Linear Regression  0.658180 10.120416 3.181260
    Random Forest  0.771693  6.759597 2.599923

Best Model: Random Forest
```

In [ ]: